APLICAÇÕES COM COMUNICAÇÃO SEM FIO PARA IoT – MQTT
(TAREFA 2 DA UNIDADE 2, CAPÍTULO 3)

Aluno: Alan Sovano Gomes
Matrícula: 202420110940138
Trilha: Software Embarcado
Mentora: Gabriela Teixeira

1. Enunciados

- **Questão (1):** Utilizando a placa Bitdoglab, crie um programa para monitorar o status de um pino, que possa ser definido como entrada, da placa Bitdoglab, e enviar, a cada 1 segundo, o status atual para o servidor HiveMQ, utilizando o protocolo MQTT. Além disso, como desafio extra, acrescente algum sensor e envie a informação desse sensor para o servidor.

- **Questão (2):** Utilizando a placa Bitdoglab, crie um programa para receber dados do servidor HiveMQ, utilizando o protocolo MQTT. Os dados recebidos devem ligar ou desligar um LED, que deve ser ligado a placa. Além disso, como desafio extra, implemente um LCD ou outro tipo de display para monitorar o recebimento de mensagens.

2. Considerações iniciais

Para que os problemas propostos fossem resolvidos, foi elaborado um código único para a placa BitDogLab, em linguagem C, capaz de estabelecer uma comunicação TCP/IP com a rede local para, em seguida, acessar um servidor (*broker*) MQTT e iniciar a troca de informações. A partir disso, os seguintes pontos foram definidos com relação ao comportamento da BitDogLab enquanto cliente MQTT:

-

- A placa deve acessar o servidor HiveMQ para iniciar o protocolo MQTT, com a comunicação sendo monitorada pelo programa MQTTX;
- Ao estabelecer com sucesso a comunicação via MQTT, o LED associado ao módulo CYW43 da Raspberry Pi Pico W deve ser ligado (funcionando como indicador visual do início da troca de dados);
- O identificador (ID) da placa durante a execução protocolo deve ser "Pico_W_Alan_Sovano";
- A BitDogLab deve enviar o estado do botão A (se pressionado ou não) e o valor lido pelo sensor de temperatura interno da placa (em graus Celsius) a cada um segundo. Essas informações devem ser publicadas no tópico "pico_w/Alan_Sovano/pub";
- A BitDogLab deve assinar o tópico "pico_w/Alan_Sovano/recv" para receber e processar as mensagens que chegam nele. Caso a mensagem seja "liga" ou "desliga", o LED RGB da placa deve ligar ou desligar na cor azul;
- As mensagens recebidas no tópico assinado devem ser mostradas no *display* OLED da BitDogLab em tempo real.

Com a definição do que o protocolo deve gerenciar, foi possível definir os requisitos básicos para o código. De forma geral, ele deve ser capaz de manipular as entradas e saídas (E/S) digitais do sistema, utilizar o conversor analógico-digital (ADC), estabelecer uma comunicação I2C com o *display* OLED e manipular *strings* durante o envio e o recebimento de mensagens através do protocolo MQTT. Para a manipulação do *display* OLED, foi utilizada uma biblioteca disponível no GitHub, a qual implementa funções básicas para mostrar textos no periférico. A biblioteca está disponível em: https://github.com/daschr/pico-ssd1306.git.

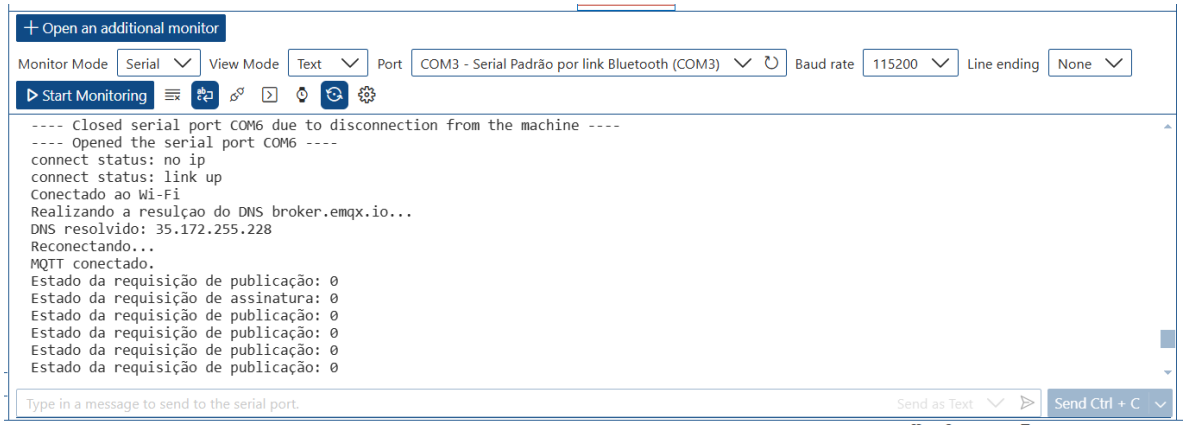
.

3. Comentários gerais sobre o código construído

O primeiro passo realizado para a construção do código foi estabelecer uma sequência de funções capazes de iniciar e manter a conexão Wi-Fi e o protocolo MQTT funcionando corretamente. Ambos os protocolos foram gerenciados pela biblioteca lwIP (*lightweight IP*), frequentemente utilizada para configurar e estabelecer vários tipos de comunicação sem fio em sistemas embarcados. A biblioteca está disponível em: https://www.nongnu.org/lwip/2_0_x/index.html. Vale ressaltar, ainda, que durante a execução

do código, várias mensagens são enviadas pela porta USB para a depuração de possíveis erros (conforme ilustrado na Figura 1).

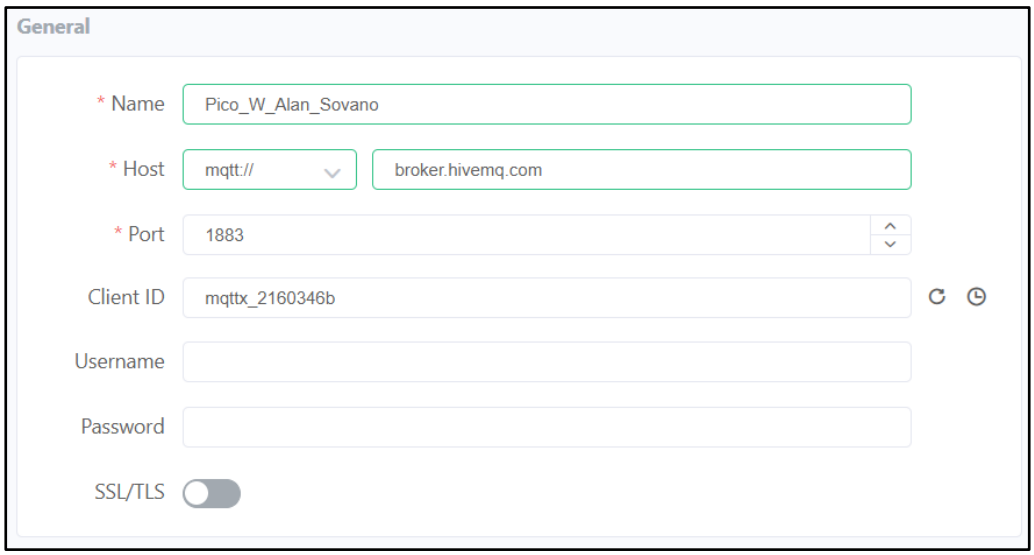
Figura 1 – Envio de mensagens pela BitDogLab ao computador para depuração do código.



Fonte: Autor (2025).

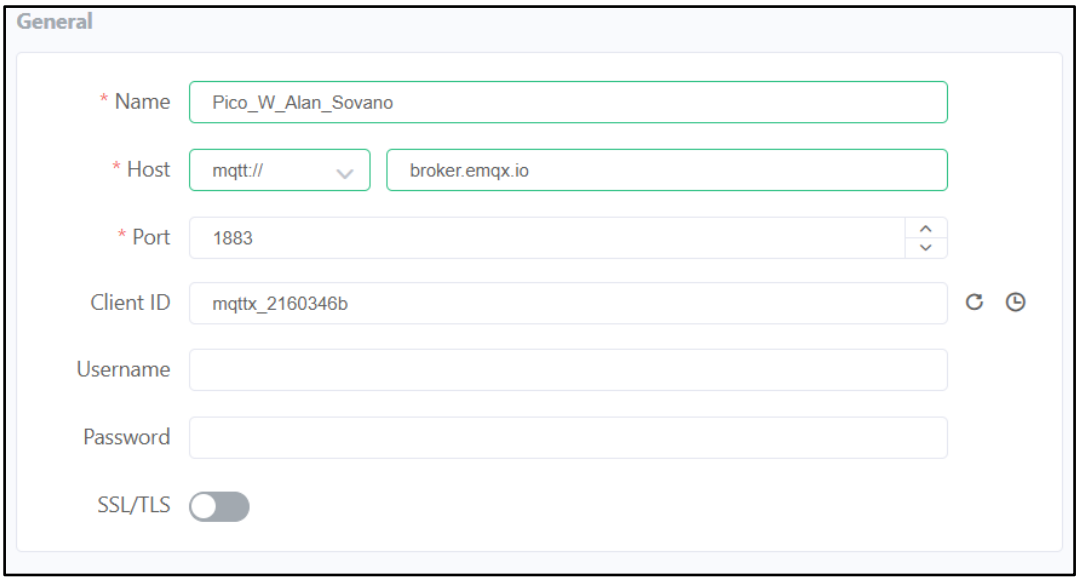
Para estabelecer a comunicação utilizando o programa MQTTX, é necessário definir o endereço do servidor e a porta de comunicação. Ambos os servidores utilizados (HiveMQ e EMQX) utilizem a porta 1883 para se comunicarem. As Figuras 2 e 3 ilustram essa etapa de configuração, com os respectivos domínios dos servidores.

Figura 2 – Configuração do MQTTX para comunicação com o servidor HiveMQ.



Fonte: Autor (2025).

Figura 3 – Configuração do MQTTX para comunicação com o servidor EMQX.



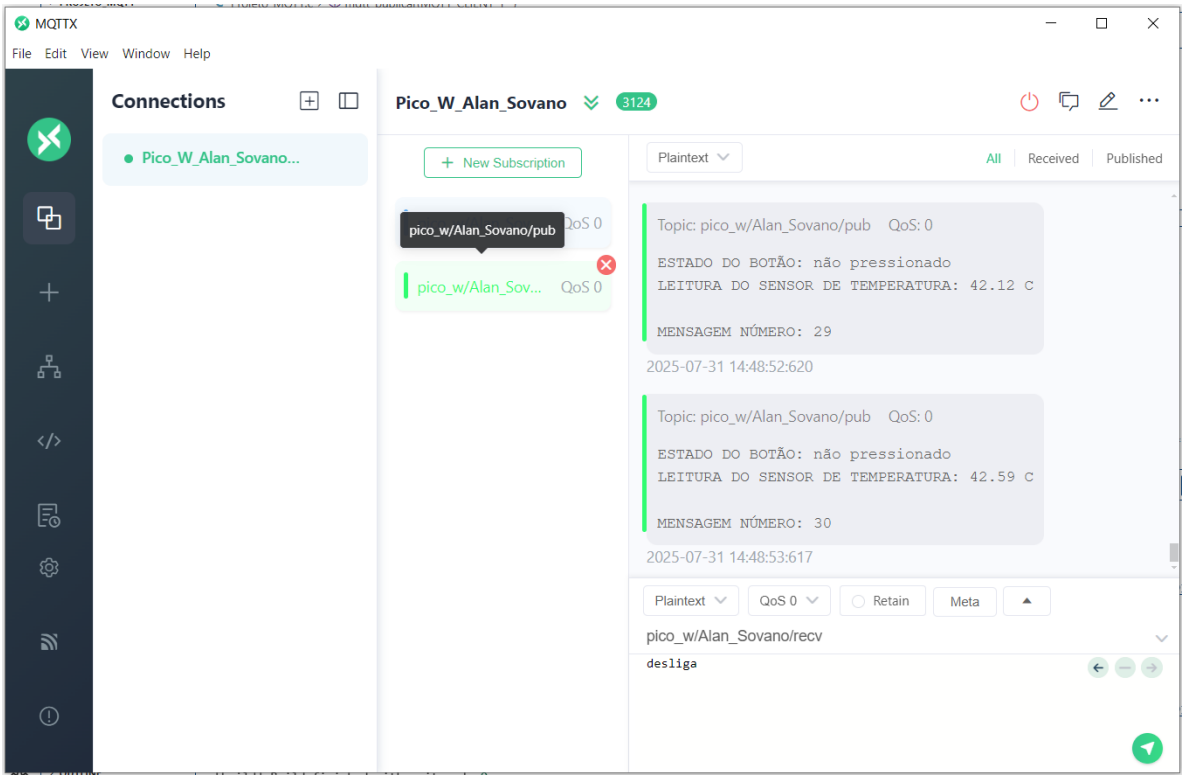
Fonte: Autor (2025).

Com a comunicação estabelecida, o formato da mensagem que é publicada a cada um segundo foi definido Para isso, optou-se por associar o estado lógico do botão a uma *string*. Quando o valor lógico do pino ligado ao Botão A é alto, a *string* "não pressionado" aparece na mensagem enviada ao tópico "pico_w/Alan_Sovano/pub" (e, de forma análoga, a *string* "pressionado" aparece quando o valor lógico do pino é baixo). Vale ressaltar que isso tem relação com a ligação interna da placa BitDogLab (o pressionar do Botão A liga uma das entradas digitais do circuito direto ao GND da placa).

Ainda sobre o processo de publicação das mensagens, o valor que chega ao conversor ADC, proveniente do sensor de temperatura, é convertido para graus Celsius e enviado junto com a informação sobre o estado do botão. Por fim, uma terceira informação é disponibilizada para visualização: a numeração da mensagem enviada (para que o usuário consiga verificar se o protocolo MQTT foi iniciado há muito ou pouco tempo). A Figura 4 ilustra a tela do programa MQTTX com a mensagem em questão sendo recebida.

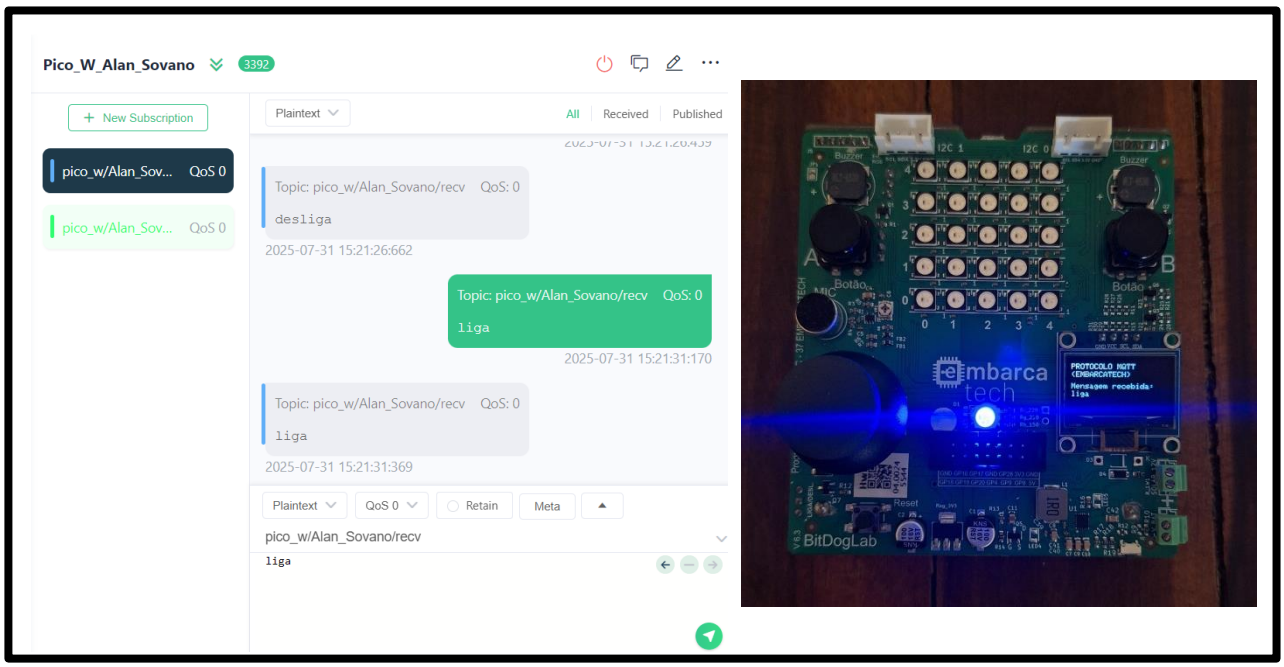
Com relação ao recebimento das mensagens pela BitDogLab, além do acionamento ou não do LED azul como descrito, deve aparecer no *display* OLED os dizeres "PROTOCOLO MQTT (EMBARCATECH)", seguidos de "MENSAGEM RECEBIDA: " e da mensagem que chega à placa. A situação na qual o LED azul fica aceso pode ser visualizada através da Figuras 5, enquanto a situação em que o LED fica apagado pode ser visualizada através dasFiguras 6.

Figura 4 – Envio de mensagens da BitDogLab para o servidor MQTT.



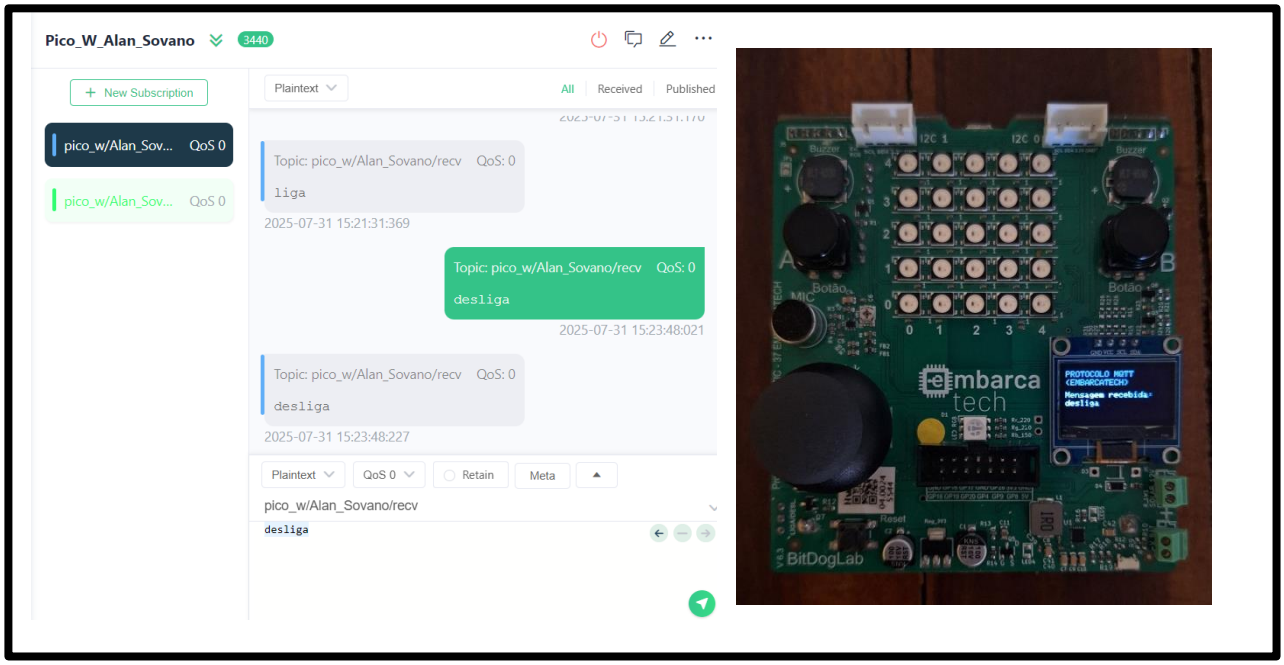
Fonte: Autor (2025).

Figura 5 – Envio de mensagem para BitDogLab pelo servidor MQTT para acender o LED.



Fonte: Autor (2025).

Figura 6 – Envio de mensagem para BitDogLab pelo servidor MQTT para apagar o LED.



Fonte: Autor (2025).

Diante do exposto, é possível confirmar que o código em C desenvolvido foi capaz de realizar os objetivos propostos pela Questão (1) e pela Questão (2). Por fim, para comprovar o funcionamento do código, o link a seguir leva para uma pasta no Google Drive que possui os arquivos do projeto desenvolvido (arquivo principal e bibliotecas auxiliares), bem como dois vídeos do sistema funcionando na placa BitDogLab (tanto com o servidor HiveMQ quanto o servidor EMQX):

https://drive.google.com/drive/folders/1L3yGYM-2oY3FXK7MpAXRih7SFMAaVtUe?usp=sharing

4. Conclusão

A partir do desenvolvimento da atividade, foi possível aprofundar conceitos sobre Internet das Coisas (IoT) e redes de computadores: enquanto o Wi-Fi representa a camada física e enlace, o MQTT representa a camada de aplicação dentro da arquitetura de comunicação utilizada (Tanenbaum, Feamster & Wetherall, 2021). A utilização de ambos permitiu a comunicação de dispositivos distintos remotamente, sem que ambos estejam necessariamente em uma mesma rede local.

Durante o desenvolvimento da atividade, foi possível perceber a importância de se trabalhar com bibliotecas de terceiros para acelerar o andamento projeto e, diante disso, ficou clara a importância de iniciativas de código aberto como a *lightweight IP*, a qual busca ser confiável, segura e leve para aplicações embarcadas. Dessa forma, mais do que desenvolver um projeto do zero, é importante ter a habilidade de ler a documentação e entender a estrutura de tais bibliotecas, bem como os conceitos envolvidos em suas implementações (como os conceitos de redes de computadores).

Por fim, vale ressaltar que a atividade proporcionou um entendimento muito melhor sobre o que é e para que serve o protocolo MQTT, gerando um embasamento para outras aplicações futuras que envolvam a comunicação entre dispositivos de forma remota, leve e simples.

5. Código em linguagem C desenvolvido para a placa BitDogLab

```
/*
Embarcatech - Trilha de Software Embarcado - Tarefa da Unidade 2 Capítulo 3 (MQTT)
Aluno: Alan Sovano Gomes
Matrícula: 202420110940138
Mentora: Gabriela Teixeira
*/

// Definição das bibliotecas
#include <stdio.h>
#include <string.h>
#include "pico/stdlib.h"
#include "hardware/adc.h"
#include "hardware/i2c.h"
#include "pico/cyw43_arch.h"

// Biblioteca utilizada para trabalhar com o display OLED
#include "ssd1306.h"

// Biblioteca para geração de uma pilha de protocolos TCP/IP leve e uso do MQTT
#include "lwip/pbuf.h"      //
#include "lwip/tcp.h"       // Comunicação via protocolo TCP
#include "lwip/dns.h"       // Implementa DNS (transforma o nome do domínio em um
IP)
#include "lwip/apps/mqtt.h" // API para MQTT (fornece as funções básicas para
trabalhar com MQTT)
#include "lwip/altcp_tcp.h" // TCP alternativo (permite o uso de TCP com TSL/SSL
sem mudanças no código da aplicação)
#include "lwip/altcp_tls.h" // Complemento da "ALTCP" (para oferecer suporte À
TLS/SSL, permitindo conexões mais seguras)
```

```
// Macros para se trabalhar com o MQTT
#define DEBUG_printf printf // Macro para ajudar a habilitar/desabilitar "printf"
utilizados somente para debug
//#define MQTT_SERVER_HOST "broker.hivemq.com" // Endereço do servidor MQTT
(HiveMQ)
#define MQTT_SERVER_HOST "broker.emqx.io" // Endereço do servidor MQTT (EMQX)
#define MQTT_SERVER_PORT 1883 // Porta de comunicação com o servidor MQTT
#define MQTT_TLS 0 // Ativação ou não do protocolo de segurança TLS
#define BUFFER_SIZE 1024 // Tamanho do buffer para enviar/receber a mensagem
#define ID_DO_CLIENTE "Pico_W_Alan_Sovano" // ID utilizado para comunicação MQTT
#define PUB_DELAY_US 1000000 // Tempo de espera (microssegundos) para o laço de
publish do MQTT

// Configurações da rede wi-fi que será utilizada (ALTERAR AQUI SE MUDAR A
REDE!!!)
#define WIFI_SSID "A54 de Alan"
#define WIFI_PASSWORD "testehoje"

// Macros para elementos de hardware
#define LED_PIN_B 12 // Pino do led RGB (cor azul)
#define BUTTON5_PIN 5 // Pino do botão A
#define LED_PIN CYW43_WL_GPIO_LED_PIN // Pino referente ao LED do módulo Wi-Fi


// Define uma variável do tipo struct para lidar com parâmetros de cliente MQTT
typedef struct MQTT_CLIENT_T_
{
    ip_addr_t remote_addr;
    mqtt_client_t *mqtt_client;
    u32_t received;
    u32_t counter;
    u32_t reconnect;
} MQTT_CLIENT_T;


// Variáveis globais
static uint32_t counter = 0; // Contador de mensagens
static uint32_t last_time = 0; // Variável auxiliar para o tempo de publish do
MQTT
bool estado_logico = false; // Variável para armazenar leitura do botão
volatile absolute_time_t ultima_pressao_A = 0; // Variável para debouncing do
botão
float temperatura = 0.0;  // Variável para guardar o valor da temperatura em graus
Celsius
uint16_t raw_value; // Variável que irá receber os dados do sensor de temperatura
const float conversion_factor = 3.3f / (1 << 12); // Fator de conversão para o
valor de temperatura lido
char estado_botao_string[20] = "não pressionado"; // String que armazena o estado
lógico do botão
```

```
bool mqtt_conectado = false; // Verifica se a conexão MQTT foi realizada com
sucesso
bool mqtt_conectando = false; // Verifica se a conexão MQTT está em processo de
conexão
const char *words[]= {"PROTOCOLO MQTT", "(EMBARCATECH)","Mensagem recebida: "}; //
Strings utilizadas no display OLED



/*********Funções para processar as mensagens recebidas via MQTT*************/

// Função de callback para tratar as mensagens recebidas em um tópico MQTT no qual
a Pi Pico está inscrita
static void mqtt_pub_data_cb(void *arg, const u8_t *data, u16_t len, u8_t flags)
{
    // Criação de um buffer para receber os dados
    char buffer[BUFFER_SIZE];

    // Verifica se a mensagem cabe dentro do buffer
    if (len < BUFFER_SIZE)
    {
        // Copia a mensagem para o buffer
        memcpy(buffer, data, len);
        buffer[len] = '\0';
        DEBUG_printf("Mensagem recebida: %s\n", buffer);

        // Tomada de decisão a partir da mensagem (ligar ou desligar o LED)
        if (strcmp(buffer, "liga") == 0)
        {
            gpio_put(LED_PIN_B, true);
        }
        else if (strcmp(buffer, "desliga") == 0)
        {
            gpio_put(LED_PIN_B, false);
        }
    }
    else
    {
        // Caso a mensagem seja muito grande, mostra uma mensagem de erro
        DEBUG_printf("Mensagem muito longa, descartando.\n");
    }

    // Declarando variável utilizada para definir o display OLED
    ssd1306_t disp;
    disp.external_vcc=false;
    ssd1306_init(&disp, 128, 64, 0x3C, i2c1);
    ssd1306_clear(&disp);

    // Escrevendo a mensagem recebida via MQTT no display OLED
    ssd1306_draw_string(&disp, 0, 05, 1, words[0]);
```

```
    ssd1306_draw_string(&disp, 0, 15, 1, words[1]);
    ssd1306_draw_string(&disp, 0, 30, 1, words[2]);
    ssd1306_draw_string(&disp, 0, 40, 1, buffer);
    ssd1306_show(&disp);
    ssd1306_clear(&disp);

}

// Debug da requisição de assinatura
void mqtt_sub_request_cb(void *arg, err_t err)
{
    DEBUG_printf("Estado da requisição de assinatura: %d\n", err);
}

// Debug da inicialização da comunicação
static void mqtt_pub_start_cb(void *arg, const char *topic, u32_t tot_len)
{
    DEBUG_printf("Incoming message on topic: %s\n", topic);
}


/****************Funções para lidar com a inicialização do MQTT**************/

// Função de inicialização para alocar memória para o cliente MQTT (retorna um
ponteiro para a variável do tipo MQTT_CLIENT_T criada anteriormente)
static MQTT_CLIENT_T* mqtt_client_init(void)
{
    // Aloca a  memória para a variável struct que define o cliente MQTT
    MQTT_CLIENT_T *state = calloc(1, sizeof(MQTT_CLIENT_T));

    // Se a alocação falha, avisa e retorna um ponteiro nulo
    if (!state)
    {
        DEBUG_printf("Falha ao alocar variável 'state'\n");
        return NULL;
    }

    // Se a alocação de memória é um sucesso, cria um ponteiro para a estrutura
    return state;
}



// Função de callback para quando o IP é encontrado via DNS
void dns_found(const char *name, const ip_addr_t *ipaddr, void *callback_arg)
{
    // Recupera o ponteiro para o estado do cliente MQTT
    MQTT_CLIENT_T *state = (MQTT_CLIENT_T*)callback_arg;
    if (ipaddr)
```

```
    {
        // Salva o endereço de IP na estrutura do cliente MQTT
        state->remote_addr = *ipaddr;
        DEBUG_printf("DNS resolvido: %s\n", ip4addr_ntoa(ipaddr));
    }
    else
    {
        DEBUG_printf("Falha na resolução do DNS.\n");
    }
}


// Procura converter o nome do host do broker MQTT para um endereço de IP válido
void run_dns_lookup(MQTT_CLIENT_T *state)
{
    DEBUG_printf("Realizando a resolução do DNS %s...\n", MQTT_SERVER_HOST);

    // Utiliza a função "dns_gethostbyname()" (da biblioteca lwIP) para encontrar
o IP do host do broker
    if (dns_gethostbyname(MQTT_SERVER_HOST, &(state->remote_addr), dns_found,
state) == ERR_INPROGRESS)
    {
        while (state->remote_addr.addr == 0)
        {
            // Processa eventos da rede Wi-Fi enquanto espera a resolução do DNS
            cyw43_arch_poll();
            sleep_ms(10);
        }
    }
}


// Função de callback para teste da conexão MQTT
static void mqtt_conexao_cb(mqtt_client_t *client, void *arg,
mqtt_connection_status_t status)
{
    // Iniciou o processo de conexão
    mqtt_conectando = false;
    MQTT_CLIENT_T *state = (MQTT_CLIENT_T *)arg;  // <- pega o contexto passado

    // No caso de conexão com sucesso, acende o led verde do módulo WI-FI
    if (status == MQTT_CONNECT_ACCEPTED)
    {
        mqtt_conectado = true;
        cyw43_arch_gpio_put(LED_PIN, 1);
        DEBUG_printf("MQTT conectado.\n");
```

```
        // Se ocorre a conexão com o servidor MQTT, faz a inscrição nos tópicos
descritos neste trecho
        mqtt_set_inpub_callback(state->mqtt_client, mqtt_pub_start_cb,
mqtt_pub_data_cb, NULL);
        mqtt_sub_unsub(state->mqtt_client, "pico_w/Alan_Sovano/recv", 0,
mqtt_sub_request_cb, NULL, 1);

    }
    else
    {
         mqtt_conectado = false;
        cyw43_arch_gpio_put(LED_PIN, 0);
        DEBUG_printf("A conexão com o MQTT falhou.\n");

    }
}

// Função para realizar a conexão MQTT
err_t mqtt_conectar(MQTT_CLIENT_T *state)
{
    struct mqtt_connect_client_info_t ci = {0};
    ci.client_id = ID_DO_CLIENTE;
    return mqtt_client_connect(state->mqtt_client, &(state->remote_addr),
MQTT_SERVER_PORT, mqtt_conexao_cb, state, &ci);
}



/***************Funções para fazer a publicação das mensagens****************/

// Debug da requisição de publicação
void mqtt_pub_request_cb(void *arg, err_t err)
{
    DEBUG_printf("Estado da requisição de publicação: %d\n", err);
}


// Função de publicação do MQTT
err_t mqtt_publicar(MQTT_CLIENT_T *state)
{

    // Verifica se passou 1 segundo
    if(absolute_time_diff_us(last_time, get_absolute_time()) >= PUB_DELAY_US)
    {
        last_time = get_absolute_time();
        char buffer[BUFFER_SIZE];
        counter += 1;

        // Lê o estado lógico do botão (com debouncing de 100 ms)
        if(absolute_time_diff_us(ultima_pressao_A, get_absolute_time()) >= 100000)
```

```
        {
            estado_logico = gpio_get(BUTTON5_PIN);
            ultima_pressao_A = get_absolute_time();
        }

        // Escrevendo o estado do botão como texto
        if(estado_logico==1)
            {
                strcpy(estado_botao_string,"não pressionado");
            }
        else
            {
                strcpy(estado_botao_string,"pressionado");
            }


        // Leitura da temperatura interna
        adc_select_input(4);
        raw_value = adc_read();
        temperatura = 27.0f - ((raw_value * conversion_factor) - 0.706f) /
0.001721f;

        // Constrói a mensagem para enviar via MQTT
        snprintf(buffer, BUFFER_SIZE,
            "ESTADO DO BOTÃO: %s \n"
            "LEITURA DO SENSOR DE TEMPERATURA: %.2f C\n\n"
            "MENSAGEM NÚMERO: %u",
             estado_botao_string, temperatura, counter);
        return mqtt_publish(state->mqtt_client, "pico_w/Alan_Sovano/pub", buffer,
strlen(buffer), 0, 0, mqtt_pub_request_cb, state);
    }
    else
    {
        return 0;
    }
}



/************************Criação do void setup()***************************/

// Configurações iniciais do microcontrolador (inicialização do botão, ADC e LED
utilizado para debug)
void setup()
{
        // Inicialização da comunicação padrão do Pi Pico
        stdio_init_all();

        //Comunicação I2C
        i2c_init(i2c1, 400000);
```

```
        gpio_set_function(14, GPIO_FUNC_I2C);
        gpio_set_function(15, GPIO_FUNC_I2C);
        gpio_pull_up(14);
        gpio_pull_up(15);

        // Inicializando o pino do botão
        gpio_init(BUTTON5_PIN);
        gpio_set_dir(BUTTON5_PIN, GPIO_IN);
        gpio_pull_up(BUTTON5_PIN);

        // Inicializando o pino do LED
        gpio_init(LED_PIN_B);
        gpio_set_dir(LED_PIN_B, GPIO_OUT);

        // Inicializa o ADC
        adc_init();
        adc_set_temp_sensor_enabled(true);

        // Inicializa o display para desligá-lo (caso estivesse ligado
anteriormente)
        ssd1306_t disp;
        disp.external_vcc=false;
        ssd1306_init(&disp, 128, 64, 0x3C, i2c1);
        ssd1306_poweroff(&disp);

}


/******************* Função principal do código ****************************/
int main()
{

    // Realiza as configurações iniciais
    setup();


    //Inicializa o chip wi-fi do raspberry pi pico W
     while (cyw43_arch_init())
    {
        printf("Falha ao inicializar Wi-Fi\n");
        sleep_ms(100);
        return -1;
    }

    // Colocando o pi pico W em modo station (vai se conectar ao roteador wi-fi)
    cyw43_arch_enable_sta_mode();

    // Tenta se conectar à rede wi-fi por 1 minuto
    printf("Conectando ao Wi-Fi...\n");
```

```
    while (cyw43_arch_wifi_connect_timeout_ms(WIFI_SSID, WIFI_PASSWORD,
CYW43_AUTH_WPA2_AES_PSK, 60000))
    {
        printf("Falha ao conectar à rede Wi-Fi\n");
        sleep_ms(100);
        return -1;
    }
    printf("Conectado ao Wi-Fi\n");

    MQTT_CLIENT_T *state = mqtt_client_init();
    run_dns_lookup(state);
    state->mqtt_client = mqtt_client_new();

    // Realiza a conexão MQTT e entra no laço principal do programa
    if(mqtt_conectar(state) == ERR_OK)
    {

        while(true)
        {
            cyw43_arch_poll();

            if(mqtt_client_is_connected(state->mqtt_client) && mqtt_conectado)
            {
                // Realizar a publicação desejada no tópico anteriormente definido
                mqtt_publicar(state);
                sleep_ms(50);
            }
            else if (!mqtt_conectando)
            {
                DEBUG_printf("Reconectando...\n");
                mqtt_conectando = true;
                cyw43_arch_gpio_put(LED_PIN, 0);

                // Tenta reconectar
                mqtt_conectar(state);
                sleep_ms(250);
            }
            else
            {
                DEBUG_printf("Aguarde...\n");
                sleep_ms(100);
            }

        }
    }

    cyw43_arch_deinit();
    return 0;

}
```

6. Referências

Tanenbaum, A.S.; Feamster, N.; Wetherall, D. **Redes de Computadores**. 6 ed. São Paulo: Pearson, 2021.